

Apurba Koirala

22BCE3799

Guided by: Dr. Sharmila C

Deep Learning Lab Assessment-1

Lab Slot L9+ L10

Question no 1

You have recently joined a company as a machine learning intern. Your task is to develop a simple binary emotion classifier using EEG signals recorded from two electrodes, representing Feature 1 and Feature 2. The company provides labeled training data from two emotional states:

- Relaxed: [2, 3], [1, 4], [2, 4]
- Stressed: [4, 2], [5, 1], [4, 3]

The emotional states are labeled as follows:

- Relaxed → Class = +1
- Stressed → Class = -1

Write a Python program to:

- Implement the perceptron learning algorithm with step-by-step weight updates.
- Visualize the errors across epochs.
- Demonstrate convergence of weights when applied to a linearly separable dataset.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [2]: X = [
    [2, 3], # Relaxed
    [1, 4], # Relaxed
    [2, 4], # Relaxed
    [4, 2], # Stressed
    [5, 1], # Stressed
    [4, 3], # Stressed
]
```

```
In [3]: y = [1, 1, 1, -1, -1, -1]
```

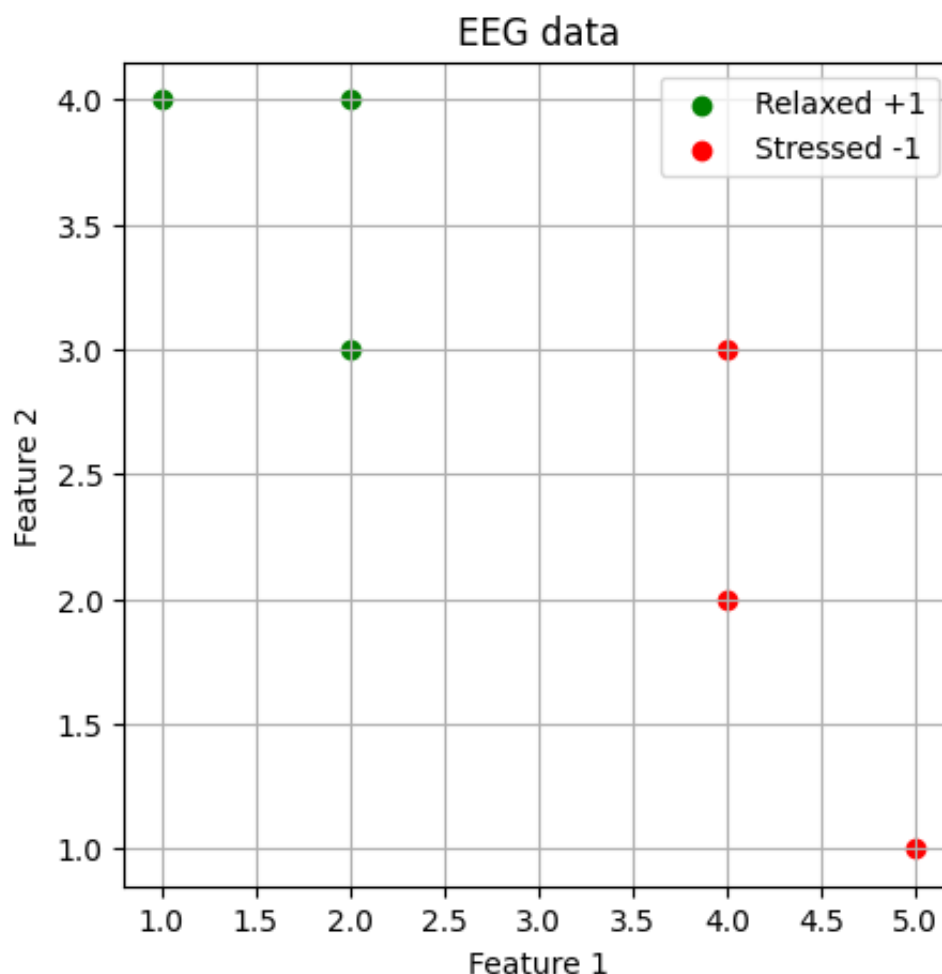
```
In [4]: X_pos = [X[i] for i in range(len(X)) if y[i] == 1] # Relaxed
X_neg = [X[i] for i in range(len(X)) if y[i] == -1] # Stressed
```

```
In [5]: x1_pos = [pt[0] for pt in X_pos]
x2_pos = [pt[1] for pt in X_pos]
x1_neg = [pt[0] for pt in X_neg]
x2_neg = [pt[1] for pt in X_neg]
```

```
In [6]: x1_pos
```

```
Out[6]: [2, 1, 2]
```

```
In [7]: plt.figure(figsize = (5,5))
plt.scatter(x1_pos, x2_pos, color = 'green', label = "Relaxed +1")
plt.scatter(x1_neg, x2_neg, color = 'red', label = "Stressed -1")
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.grid(True)
plt.title("EEG data")
plt.show()
```



```
In [8]: def predict (x, w, b):
result = w[0] * x[0] + w[1] * x[1] + b
```

```

if result >= 0:
    return 1
else:
    return -1

```

```

In [9]: w = [0, 0] # weights
        b = 0      # bias
        eta = 1    # learning rate
        max_epochs = 100
        errors_per_epoch = []

```

```

In [10]: for epoch in range(max_epochs):
          errors = 0
          for i in range(len(X)):
              y_pred = predict(X[i], w, b)
              if y_pred != y[i]:
                  # Update weights and bias
                  w[0] += eta * y[i] * X[i][0]
                  w[1] += eta * y[i] * X[i][1]
                  b += eta * y[i]
                  errors += 1
          errors_per_epoch.append(errors)

          # Early stopping if converged
          if errors == 0:
              print(f"Converged after {epoch+1} epochs. Final weights {w}")
              break

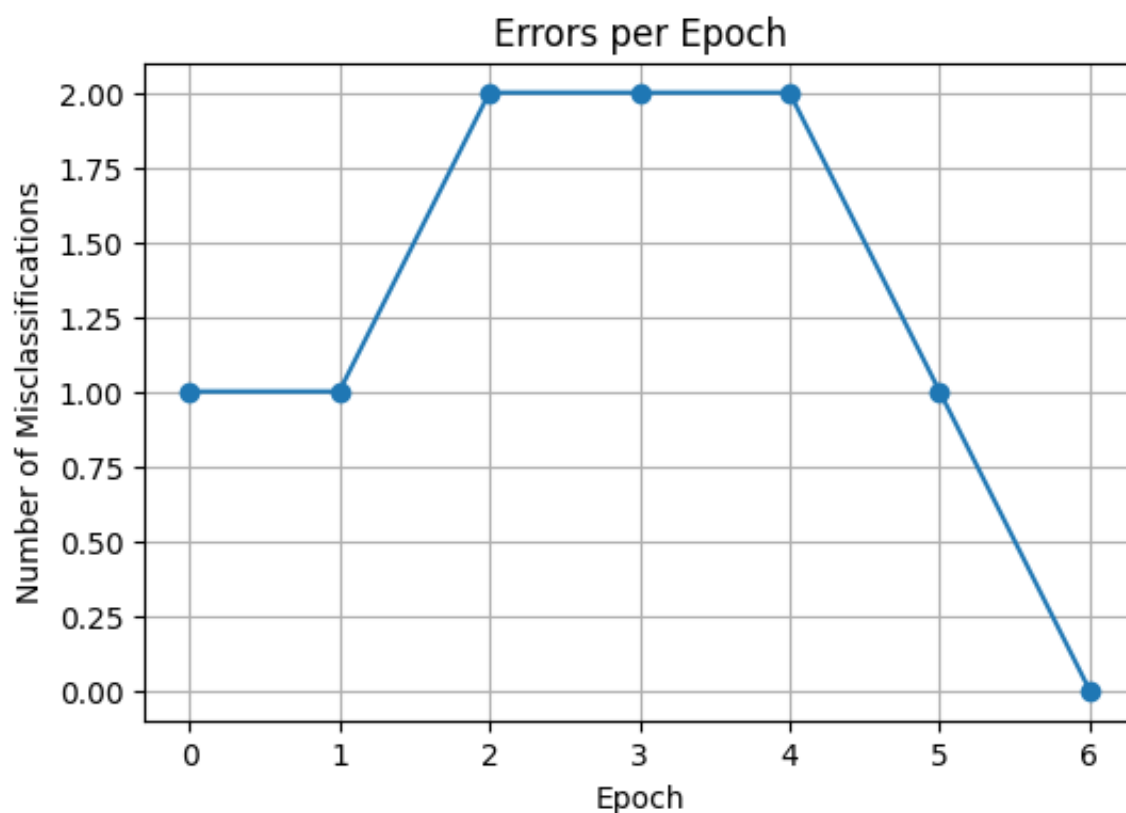
```

Converged after 7 epochs. Final weights [-6, 6] and final bias 1

```

In [11]: plt.figure(figsize=(6, 4))
          plt.plot(errors_per_epoch, marker='o')
          plt.title("Errors per Epoch")
          plt.xlabel("Epoch")
          plt.ylabel("Number of Misclassifications")
          plt.grid(True)
          plt.show()

```

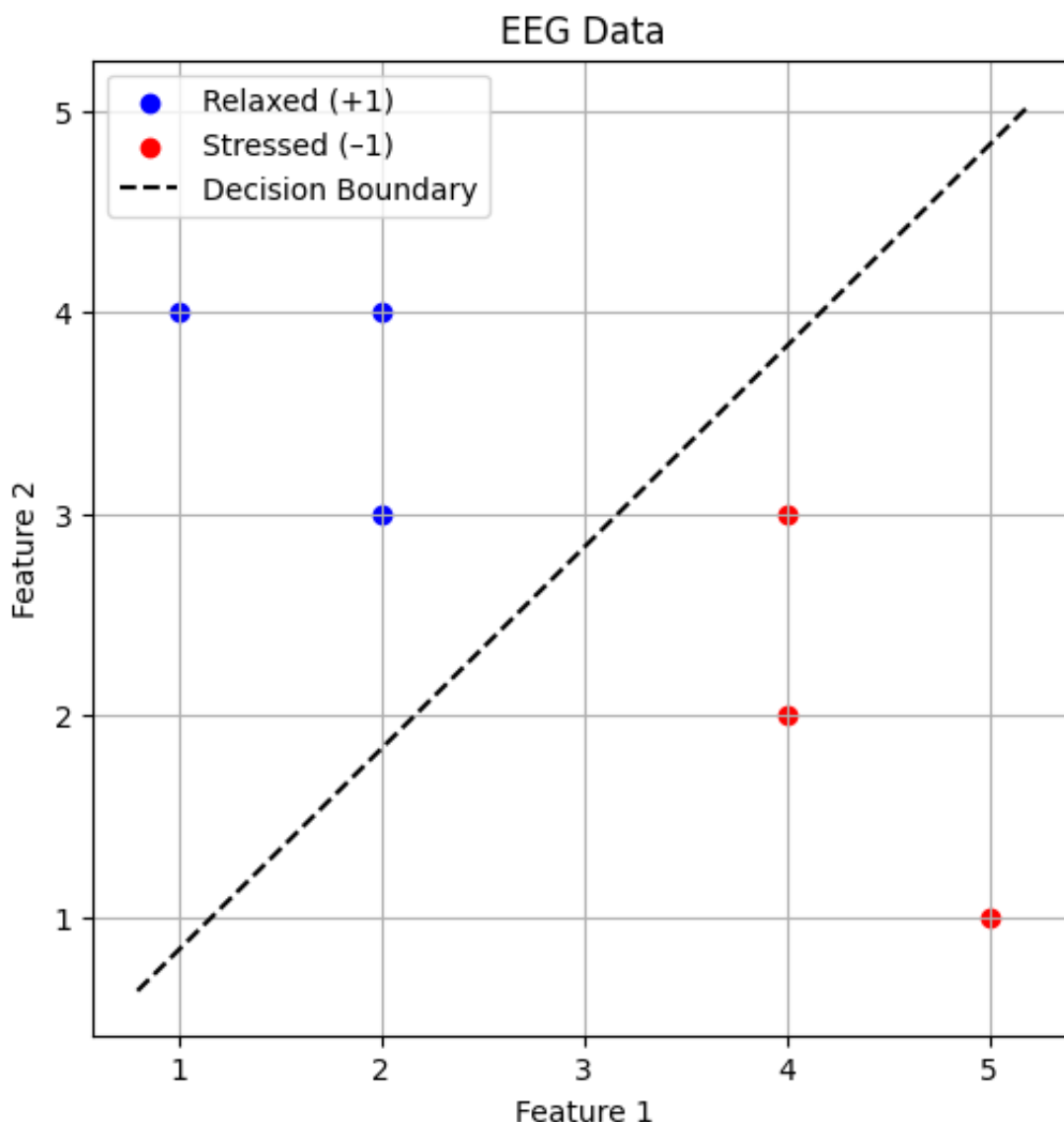


```
In [12]: plt.figure(figsize=(6,6))
plt.scatter(x1_pos, x2_pos, color='blue', label='Relaxed (+1)')
plt.scatter(x1_neg, x2_neg, color='red', label='Stressed (-1)')

x_vals = np.array(plt.gca().get_xlim()) # get current x-axis limit
y_vals = -(w[0]/w[1]) * x_vals - b/w[1] # compute y from the bound

plt.plot(x_vals, y_vals, 'k--', label='Decision Boundary')

plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('EEG Data')
plt.legend()
plt.grid(True)
plt.show()
```



Question 2

You are working with a healthcare startup building an AI-powered assistant that helps doctors identify whether a patient is likely experiencing flu symptoms or allergic reactions—two conditions that often overlap in early presentations. The sensor inputs include body temperature and nasal inflammation readings from smart diagnostic patches. Unfortunately, the data distribution is linearly inseparable due to physiological variability among patients (e.g. some allergic patients also have high temperatures). To increase diagnostic accuracy, your task is to build a Multilayer Feedforward Neural Network that learns complex boundaries and confidently distinguishes between the two classes. Demonstrate how an MLFFNN can correctly classify non-linearly separable patterns (Flu vs Allergy) using symptom intensity as features in a binary class setup with gradient descent weight updation policy.

Temperature	Inflammation	Diagnosis
98.6	0.2	Allergy (0)
99.5	0.9	Flu (1)
98.7	0.8	Flu (1)
99.0	0.3	Allergy (0)
100.2	0.7	Flu (1)
98.4	0.6	Allergy (0)

Lab Tasks

Build an MLFFNN with:

- 2 input neurons (features)
- One hidden layer (with 3–5 neurons using ReLU or tanh)
- Output layer with a sigmoid neuron
- Train using backpropagation and binary cross-entropy loss
- Track convergence metrics, prediction accuracy, and visualize the feature transformation.

```
In [13]: X = np.array([[98.6, 0.2], [99.5, 0.9], [98.7, 0.8], [99.0, 0.3], [100.2, 0.7], [98.4, 0.6]])
```

```
In [14]: y = np.array([0, 1, 1, 0, 1, 0])
```

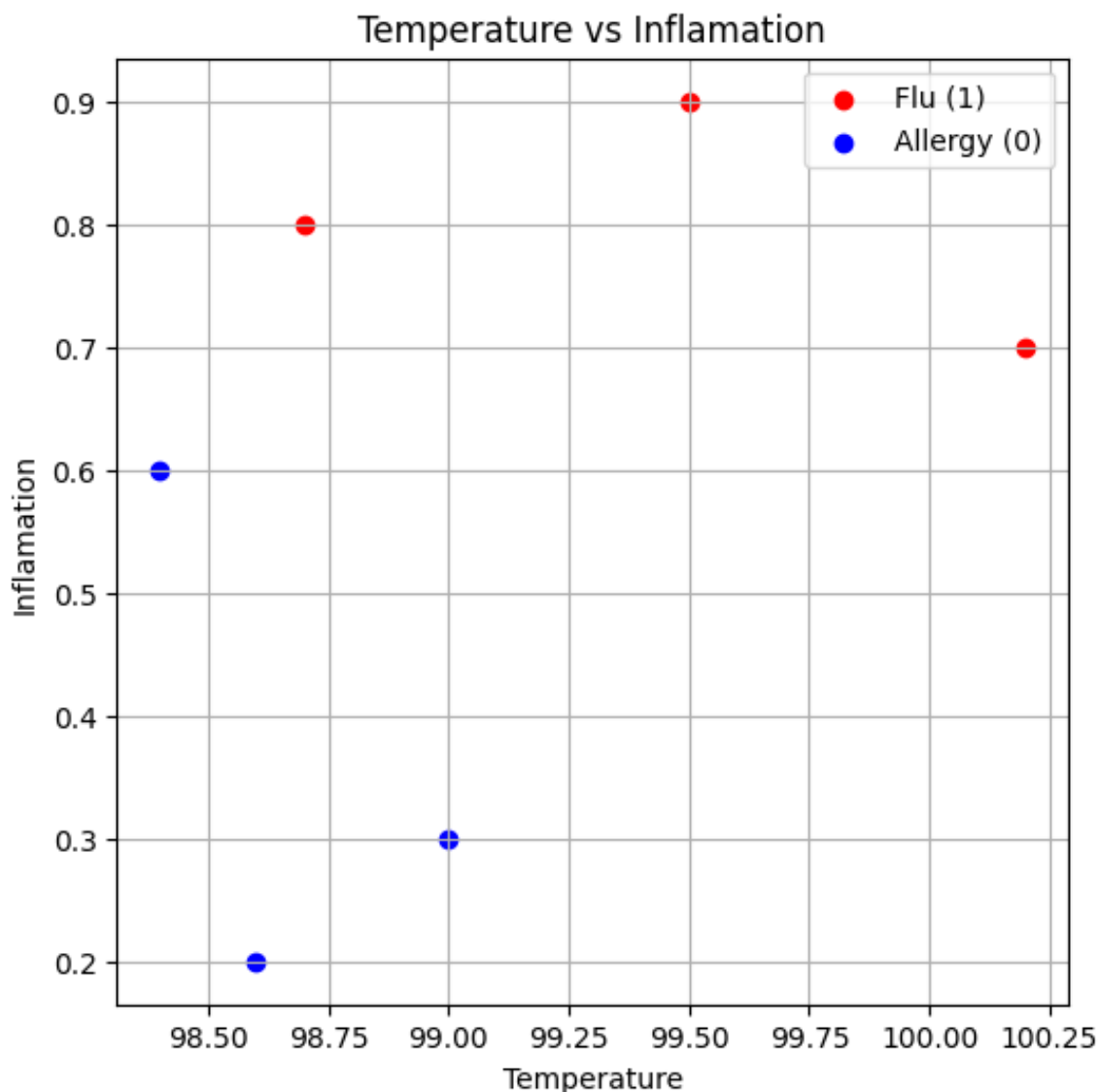
```
In [15]: X_flu = X[y==1]
X_allergy = X[y==0]
```

```
In [16]: plt.figure(figsize=(6, 6))

plt.scatter(X_flu[:, 0], X_flu[:, 1], color='red', label='Flu (1)')
plt.scatter(X_allergy[:, 0], X_allergy[:, 1], color='blue', label='Allergy (0)')

plt.xlabel('Temperature')
plt.ylabel('Inflammation')

plt.grid(True)
plt.legend()
plt.title('Temperature vs Inflammation')
plt.show()
```



```
In [17]: from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
```

```
In [18]: X_scaled = StandardScaler().fit_transform(X)
```

```
In [19]: X_scaled
```

```
Out[19]: array([[ -0.75814251, -1.5067801 ],
 [  0.70398947,  1.24473139],
 [-0.5956834 ,  0.85165832],
 [-0.10830607, -1.11370703],
 [ 1.84120323,  0.45858525],
 [-1.08306072,  0.06551218]])
```

```
In [20]: import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
```

```
In [21]: model = models.Sequential([
    layers.Input(shape=(2,)),
    layers.Dense(5, activation = 'relu'),
    layers.Dense(1, activation = 'sigmoid')
```

```
])
```

```
In [22]: model.compile(optimizer='adam', loss = 'binary_crossentropy', metri
```

```
In [23]: history = model.fit(X_scaled, y, epochs = 100, verbose = 1)
```

Epoch 1/100

1/1 ————— 1s 1s/step – accuracy: 0.6667 – loss: 0.6159

Epoch 2/100

1/1 ————— 0s 35ms/step – accuracy: 0.6667 – loss: 0.6145

Epoch 3/100

1/1 ————— 0s 56ms/step – accuracy: 0.6667 – loss: 0.6131

Epoch 4/100

1/1 ————— 0s 58ms/step – accuracy: 0.6667 – loss: 0.6117

Epoch 5/100

1/1 ————— 0s 59ms/step – accuracy: 0.6667 – loss: 0.6104

Epoch 6/100

1/1 ————— 0s 57ms/step – accuracy: 0.6667 – loss: 0.6090

Epoch 7/100

1/1 ————— 0s 58ms/step – accuracy: 0.6667 – loss: 0.6076

Epoch 8/100

1/1 ————— 0s 57ms/step – accuracy: 0.6667 – loss: 0.6062

Epoch 9/100

1/1 ————— 0s 37ms/step – accuracy: 0.6667 – loss: 0.6048

Epoch 10/100

1/1 ————— 0s 60ms/step – accuracy: 0.6667 – loss: 0.6034

Epoch 11/100

1/1 ————— 0s 33ms/step – accuracy: 0.6667 – loss: 0.6021

Epoch 12/100

1/1 ————— 0s 34ms/step – accuracy: 0.6667 – loss: 0.6007

Epoch 13/100

1/1 ————— 0s 35ms/step – accuracy: 0.6667 – loss: 0.5993

Epoch 14/100

1/1 ————— 0s 33ms/step – accuracy: 0.6667 – loss: 0.5980

Epoch 15/100

1/1 ————— 0s 58ms/step – accuracy: 0.6667 – loss: 0.5966

Epoch 16/100

1/1 ————— 0s 61ms/step – accuracy: 0.6667 – loss: 0.5952

Epoch 17/100

1/1 ————— 0s 34ms/step – accuracy: 0.6667 – loss: 0.5

939
Epoch 18/100
1/1  0s 59ms/step - accuracy: 0.6667 - loss: 0.5
925
Epoch 19/100
1/1  0s 68ms/step - accuracy: 0.6667 - loss: 0.5
912
Epoch 20/100
1/1  0s 57ms/step - accuracy: 0.6667 - loss: 0.5
898
Epoch 21/100
1/1  0s 53ms/step - accuracy: 0.6667 - loss: 0.5
885
Epoch 22/100
1/1  0s 69ms/step - accuracy: 0.6667 - loss: 0.5
871
Epoch 23/100
1/1  0s 50ms/step - accuracy: 0.6667 - loss: 0.5
858
Epoch 24/100
1/1  0s 50ms/step - accuracy: 0.6667 - loss: 0.5
845
Epoch 25/100
1/1  0s 51ms/step - accuracy: 0.6667 - loss: 0.5
831
Epoch 26/100
1/1  0s 55ms/step - accuracy: 0.6667 - loss: 0.5
818
Epoch 27/100
1/1  0s 49ms/step - accuracy: 0.6667 - loss: 0.5
805
Epoch 28/100
1/1  0s 59ms/step - accuracy: 0.8333 - loss: 0.5
791
Epoch 29/100
1/1  0s 58ms/step - accuracy: 0.8333 - loss: 0.5
778
Epoch 30/100
1/1  0s 50ms/step - accuracy: 0.8333 - loss: 0.5
765
Epoch 31/100
1/1  0s 49ms/step - accuracy: 0.8333 - loss: 0.5
752
Epoch 32/100
1/1  0s 58ms/step - accuracy: 0.8333 - loss: 0.5
739
Epoch 33/100
1/1  0s 37ms/step - accuracy: 0.8333 - loss: 0.5
726
Epoch 34/100
1/1  0s 37ms/step - accuracy: 0.8333 - loss: 0.5
712
Epoch 35/100
1/1  0s 46ms/step - accuracy: 0.8333 - loss: 0.5
699
Epoch 36/100

1/1 ————— 0s 58ms/step - accuracy: 0.8333 - loss: 0.5
686
Epoch 37/100
1/1 ————— 0s 34ms/step - accuracy: 0.8333 - loss: 0.5
673
Epoch 38/100
1/1 ————— 0s 59ms/step - accuracy: 0.8333 - loss: 0.5
660
Epoch 39/100
1/1 ————— 0s 36ms/step - accuracy: 0.8333 - loss: 0.5
647
Epoch 40/100
1/1 ————— 0s 35ms/step - accuracy: 0.8333 - loss: 0.5
635
Epoch 41/100
1/1 ————— 0s 39ms/step - accuracy: 0.8333 - loss: 0.5
622
Epoch 42/100
1/1 ————— 0s 53ms/step - accuracy: 0.8333 - loss: 0.5
609
Epoch 43/100
1/1 ————— 0s 34ms/step - accuracy: 0.8333 - loss: 0.5
596
Epoch 44/100
1/1 ————— 0s 46ms/step - accuracy: 0.8333 - loss: 0.5
583
Epoch 45/100
1/1 ————— 0s 58ms/step - accuracy: 0.8333 - loss: 0.5
570
Epoch 46/100
1/1 ————— 0s 140ms/step - accuracy: 0.8333 - loss: 0.
5558
Epoch 47/100
1/1 ————— 0s 53ms/step - accuracy: 0.8333 - loss: 0.5
545
Epoch 48/100
1/1 ————— 0s 57ms/step - accuracy: 0.8333 - loss: 0.5
532
Epoch 49/100
1/1 ————— 0s 60ms/step - accuracy: 0.8333 - loss: 0.5
519
Epoch 50/100
1/1 ————— 0s 49ms/step - accuracy: 0.8333 - loss: 0.5
507
Epoch 51/100
1/1 ————— 0s 57ms/step - accuracy: 0.8333 - loss: 0.5
494
Epoch 52/100
1/1 ————— 0s 47ms/step - accuracy: 0.8333 - loss: 0.5
482
Epoch 53/100
1/1 ————— 0s 53ms/step - accuracy: 0.8333 - loss: 0.5
469
Epoch 54/100
1/1 ————— 0s 58ms/step - accuracy: 0.8333 - loss: 0.5
456

Epoch 55/100
1/1  0s 57ms/step - accuracy: 0.8333 - loss: 0.5
444

Epoch 56/100
1/1  0s 52ms/step - accuracy: 0.8333 - loss: 0.5
431

Epoch 57/100
1/1  0s 57ms/step - accuracy: 0.8333 - loss: 0.5
419

Epoch 58/100
1/1  0s 69ms/step - accuracy: 0.8333 - loss: 0.5
406

Epoch 59/100
1/1  0s 133ms/step - accuracy: 0.8333 - loss: 0.
5394

Epoch 60/100
1/1  0s 54ms/step - accuracy: 0.8333 - loss: 0.5
382

Epoch 61/100
1/1  0s 46ms/step - accuracy: 0.8333 - loss: 0.5
369

Epoch 62/100
1/1  0s 58ms/step - accuracy: 0.8333 - loss: 0.5
357

Epoch 63/100
1/1  0s 58ms/step - accuracy: 0.8333 - loss: 0.5
344

Epoch 64/100
1/1  0s 51ms/step - accuracy: 0.8333 - loss: 0.5
332

Epoch 65/100
1/1  0s 60ms/step - accuracy: 0.8333 - loss: 0.5
320

Epoch 66/100
1/1  0s 52ms/step - accuracy: 0.8333 - loss: 0.5
308

Epoch 67/100
1/1  0s 50ms/step - accuracy: 0.8333 - loss: 0.5
295

Epoch 68/100
1/1  0s 60ms/step - accuracy: 0.8333 - loss: 0.5
283

Epoch 69/100
1/1  0s 51ms/step - accuracy: 0.8333 - loss: 0.5
271

Epoch 70/100
1/1  0s 57ms/step - accuracy: 0.8333 - loss: 0.5
259

Epoch 71/100
1/1  0s 54ms/step - accuracy: 0.8333 - loss: 0.5
246

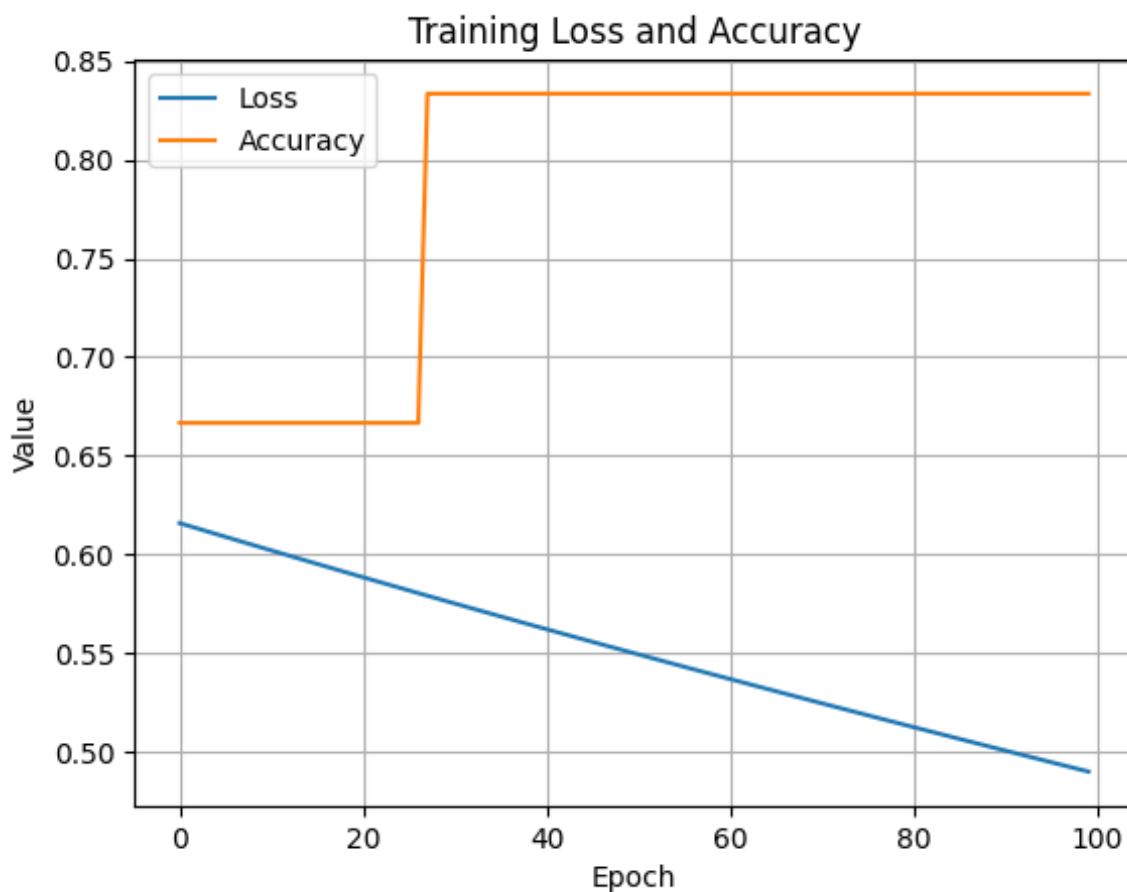
Epoch 72/100
1/1  0s 49ms/step - accuracy: 0.8333 - loss: 0.5
234

Epoch 73/100
1/1  0s 53ms/step - accuracy: 0.8333 - loss: 0.5

222
Epoch 74/100
1/1  0s 60ms/step - accuracy: 0.8333 - loss: 0.5
210
Epoch 75/100
1/1  0s 61ms/step - accuracy: 0.8333 - loss: 0.5
198
Epoch 76/100
1/1  0s 57ms/step - accuracy: 0.8333 - loss: 0.5
186
Epoch 77/100
1/1  0s 51ms/step - accuracy: 0.8333 - loss: 0.5
174
Epoch 78/100
1/1  0s 38ms/step - accuracy: 0.8333 - loss: 0.5
162
Epoch 79/100
1/1  0s 42ms/step - accuracy: 0.8333 - loss: 0.5
150
Epoch 80/100
1/1  0s 35ms/step - accuracy: 0.8333 - loss: 0.5
138
Epoch 81/100
1/1  0s 58ms/step - accuracy: 0.8333 - loss: 0.5
126
Epoch 82/100
1/1  0s 36ms/step - accuracy: 0.8333 - loss: 0.5
114
Epoch 83/100
1/1  0s 38ms/step - accuracy: 0.8333 - loss: 0.5
102
Epoch 84/100
1/1  0s 56ms/step - accuracy: 0.8333 - loss: 0.5
090
Epoch 85/100
1/1  0s 36ms/step - accuracy: 0.8333 - loss: 0.5
078
Epoch 86/100
1/1  0s 57ms/step - accuracy: 0.8333 - loss: 0.5
066
Epoch 87/100
1/1  0s 36ms/step - accuracy: 0.8333 - loss: 0.5
054
Epoch 88/100
1/1  0s 58ms/step - accuracy: 0.8333 - loss: 0.5
042
Epoch 89/100
1/1  0s 39ms/step - accuracy: 0.8333 - loss: 0.5
030
Epoch 90/100
1/1  0s 58ms/step - accuracy: 0.8333 - loss: 0.5
019
Epoch 91/100
1/1  0s 37ms/step - accuracy: 0.8333 - loss: 0.5
007
Epoch 92/100

1/1 ————— 0s 41ms/step – accuracy: 0.8333 – loss: 0.4
995
Epoch 93/100
1/1 ————— 0s 36ms/step – accuracy: 0.8333 – loss: 0.4
983
Epoch 94/100
1/1 ————— 0s 62ms/step – accuracy: 0.8333 – loss: 0.4
971
Epoch 95/100
1/1 ————— 0s 44ms/step – accuracy: 0.8333 – loss: 0.4
960
Epoch 96/100
1/1 ————— 0s 38ms/step – accuracy: 0.8333 – loss: 0.4
948
Epoch 97/100
1/1 ————— 0s 57ms/step – accuracy: 0.8333 – loss: 0.4
936
Epoch 98/100
1/1 ————— 0s 59ms/step – accuracy: 0.8333 – loss: 0.4
924
Epoch 99/100
1/1 ————— 0s 37ms/step – accuracy: 0.8333 – loss: 0.4
913
Epoch 100/100
1/1 ————— 0s 35ms/step – accuracy: 0.8333 – loss: 0.4
901

```
In [24]: plt.plot(history.history['loss'], label='Loss')
plt.plot(history.history['accuracy'], label='Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Value')
plt.title('Training Loss and Accuracy')
plt.legend()
plt.grid(True)
plt.show()
```



```
In [25]: preds = model.predict(X_scaled)
print(preds) #probabilities
```

1/1 ————— 0s 133ms/step

```
[[0.17115422]
 [0.56246585]
 [0.5861282 ]
 [0.25742847]
 [0.4880488 ]
 [0.46275705]]
```

```
In [26]: pred_classes = (preds >= 0.5).astype(int)
print(pred_classes)
```

```
[[0]
 [1]
 [1]
 [0]
 [0]
 [0]]
```

Question 3

You'll train a basic feedforward neural network to approximate the function:

$$y = 3x^2 + 2x + 1$$

This clean regression task allows you to clearly observe how optimizers impact convergence, learning rate adaptation, and final accuracy

Python snippet to generate the data

Synthetic dataset

```
import numpy as np
```

```
X = np.linspace(-10, 10, 1000).reshape(-1, 1)
```

```
y = 3 * X**2 + 2 * X + 1 + np.random.normal(0, 10, X.shape)
```

Network Architecture

- Input: 1 feature
- Hidden layers: [16 → 8 neurons], ReLU activations
- Output: 1 neuron, linear activation
- Loss: Mean Squared Error (MSE)

Optimizers to Compare Train identical models with: - SGD

- SGD + Momentum
- RMSprop
- Adam

Use your own values of Epochs, Batch size and Learning rate Display the following with plots

- Loss vs Epoch plots for each optimizer
- Training time
- Final loss values
- Scatter plot of predicted vs true values

```
In [27]: X = np.linspace(-10, 10, 1000).reshape(-1, 1)
y = 3 * X**2 + 2 * X + 1 + np.random.normal(0, 10, X.shape)
```

```
In [28]: X.shape
```

```
Out[28]: (1000, 1)
```

```
In [29]: y.shape
```

```
Out[29]: (1000, 1)
```

```
In [30]: X
```

```
Out[30]: array([[ -10.          ],
                [ -9.97997998],
                [ -9.95995996],
```

[-9.93993994],
[-9.91991992],
[-9.8998999],
[-9.87987988],
[-9.85985986],
[-9.83983984],
[-9.81981982],
[-9.7997998],
[-9.77977978],
[-9.75975976],
[-9.73973974],
[-9.71971972],
[-9.6996997],
[-9.67967968],
[-9.65965966],
[-9.63963964],
[-9.61961962],
[-9.5995996],
[-9.57957958],
[-9.55955956],
[-9.53953954],
[-9.51951952],
[-9.4994995],
[-9.47947948],
[-9.45945946],
[-9.43943944],
[-9.41941942],
[-9.3993994],
[-9.37937938],
[-9.35935936],
[-9.33933934],
[-9.31931932],
[-9.2992993],
[-9.27927928],
[-9.25925926],
[-9.23923924],
[-9.21921922],
[-9.1991992],
[-9.17917918],
[-9.15915916],
[-9.13913914],
[-9.11911912],
[-9.0990991],
[-9.07907908],
[-9.05905906],
[-9.03903904],
[-9.01901902],
[-8.998999],
[-8.97897898],
[-8.95895896],
[-8.93893894],
[-8.91891892],
[-8.8988989],
[-8.87887888],
[-8.85885886],
[-8.83883884],

[-8.81881882],
[-8.7987988],
[-8.77877878],
[-8.75875876],
[-8.73873874],
[-8.71871872],
[-8.6986987],
[-8.67867868],
[-8.65865866],
[-8.63863864],
[-8.61861862],
[-8.5985986],
[-8.57857858],
[-8.55855856],
[-8.53853854],
[-8.51851852],
[-8.4984985],
[-8.47847848],
[-8.45845846],
[-8.43843844],
[-8.41841842],
[-8.3983984],
[-8.37837838],
[-8.35835836],
[-8.33833834],
[-8.31831832],
[-8.2982983],
[-8.27827828],
[-8.25825826],
[-8.23823824],
[-8.21821822],
[-8.1981982],
[-8.17817818],
[-8.15815816],
[-8.13813814],
[-8.11811812],
[-8.0980981],
[-8.07807808],
[-8.05805806],
[-8.03803804],
[-8.01801802],
[-7.997998],
[-7.97797798],
[-7.95795796],
[-7.93793794],
[-7.91791792],
[-7.8978979],
[-7.87787788],
[-7.85785786],
[-7.83783784],
[-7.81781782],
[-7.7977978],
[-7.77777778],
[-7.75775776],
[-7.73773774],
[-7.71771772],

[-7.6976977],
[-7.67767768],
[-7.65765766],
[-7.63763764],
[-7.61761762],
[-7.5975976],
[-7.57757758],
[-7.55755756],
[-7.53753754],
[-7.51751752],
[-7.4974975],
[-7.47747748],
[-7.45745746],
[-7.43743744],
[-7.41741742],
[-7.3973974],
[-7.37737738],
[-7.35735736],
[-7.33733734],
[-7.31731732],
[-7.2972973],
[-7.27727728],
[-7.25725726],
[-7.23723724],
[-7.21721722],
[-7.1971972],
[-7.17717718],
[-7.15715716],
[-7.13713714],
[-7.11711712],
[-7.0970971],
[-7.07707708],
[-7.05705706],
[-7.03703704],
[-7.01701702],
[-6.996997],
[-6.97697698],
[-6.95695696],
[-6.93693694],
[-6.91691692],
[-6.8968969],
[-6.87687688],
[-6.85685686],
[-6.83683684],
[-6.81681682],
[-6.7967968],
[-6.77677678],
[-6.75675676],
[-6.73673674],
[-6.71671672],
[-6.6966967],
[-6.67667668],
[-6.65665666],
[-6.63663664],
[-6.61661662],
[-6.5965966],

[-6.57657658],
[-6.55655656],
[-6.53653654],
[-6.51651652],
[-6.4964965],
[-6.47647648],
[-6.45645646],
[-6.43643644],
[-6.41641642],
[-6.3963964],
[-6.37637638],
[-6.35635636],
[-6.33633634],
[-6.31631632],
[-6.2962963],
[-6.27627628],
[-6.25625626],
[-6.23623624],
[-6.21621622],
[-6.1961962],
[-6.17617618],
[-6.15615616],
[-6.13613614],
[-6.11611612],
[-6.0960961],
[-6.07607608],
[-6.05605606],
[-6.03603604],
[-6.01601602],
[-5.995996],
[-5.97597598],
[-5.95595596],
[-5.93593594],
[-5.91591592],
[-5.8958959],
[-5.87587588],
[-5.85585586],
[-5.83583584],
[-5.81581582],
[-5.7957958],
[-5.77577578],
[-5.75575576],
[-5.73573574],
[-5.71571572],
[-5.6956957],
[-5.67567568],
[-5.65565566],
[-5.63563564],
[-5.61561562],
[-5.5955956],
[-5.57557558],
[-5.55555556],
[-5.53553554],
[-5.51551552],
[-5.4954955],
[-5.47547548],

[-5.45545546],
[-5.43543544],
[-5.41541542],
[-5.3953954],
[-5.37537538],
[-5.35535536],
[-5.33533534],
[-5.31531532],
[-5.2952953],
[-5.27527528],
[-5.25525526],
[-5.23523524],
[-5.21521522],
[-5.1951952],
[-5.17517518],
[-5.15515516],
[-5.13513514],
[-5.11511512],
[-5.0950951],
[-5.07507508],
[-5.05505506],
[-5.03503504],
[-5.01501502],
[-4.99499499],
[-4.97497497],
[-4.95495495],
[-4.93493493],
[-4.91491491],
[-4.89489489],
[-4.87487487],
[-4.85485485],
[-4.83483483],
[-4.81481481],
[-4.79479479],
[-4.77477477],
[-4.75475475],
[-4.73473473],
[-4.71471471],
[-4.69469469],
[-4.67467467],
[-4.65465465],
[-4.63463463],
[-4.61461461],
[-4.59459459],
[-4.57457457],
[-4.55455455],
[-4.53453453],
[-4.51451451],
[-4.49449449],
[-4.47447447],
[-4.45445445],
[-4.43443443],
[-4.41441441],
[-4.39439439],
[-4.37437437],
[-4.35435435],

[-4.33433433],
[-4.31431431],
[-4.29429429],
[-4.27427427],
[-4.25425425],
[-4.23423423],
[-4.21421421],
[-4.19419419],
[-4.17417417],
[-4.15415415],
[-4.13413413],
[-4.11411411],
[-4.09409409],
[-4.07407407],
[-4.05405405],
[-4.03403403],
[-4.01401401],
[-3.99399399],
[-3.97397397],
[-3.95395395],
[-3.93393393],
[-3.91391391],
[-3.89389389],
[-3.87387387],
[-3.85385385],
[-3.83383383],
[-3.81381381],
[-3.79379379],
[-3.77377377],
[-3.75375375],
[-3.73373373],
[-3.71371371],
[-3.69369369],
[-3.67367367],
[-3.65365365],
[-3.63363363],
[-3.61361361],
[-3.59359359],
[-3.57357357],
[-3.55355355],
[-3.53353353],
[-3.51351351],
[-3.49349349],
[-3.47347347],
[-3.45345345],
[-3.43343343],
[-3.41341341],
[-3.39339339],
[-3.37337337],
[-3.35335335],
[-3.33333333],
[-3.31331331],
[-3.29329329],
[-3.27327327],
[-3.25325325],
[-3.23323323],

[-3.21321321],
[-3.19319319],
[-3.17317317],
[-3.15315315],
[-3.13313313],
[-3.11311311],
[-3.09309309],
[-3.07307307],
[-3.05305305],
[-3.03303303],
[-3.01301301],
[-2.99299299],
[-2.97297297],
[-2.95295295],
[-2.93293293],
[-2.91291291],
[-2.89289289],
[-2.87287287],
[-2.85285285],
[-2.83283283],
[-2.81281281],
[-2.79279279],
[-2.77277277],
[-2.75275275],
[-2.73273273],
[-2.71271271],
[-2.69269269],
[-2.67267267],
[-2.65265265],
[-2.63263263],
[-2.61261261],
[-2.59259259],
[-2.57257257],
[-2.55255255],
[-2.53253253],
[-2.51251251],
[-2.49249249],
[-2.47247247],
[-2.45245245],
[-2.43243243],
[-2.41241241],
[-2.39239239],
[-2.37237237],
[-2.35235235],
[-2.33233233],
[-2.31231231],
[-2.29229229],
[-2.27227227],
[-2.25225225],
[-2.23223223],
[-2.21221221],
[-2.19219219],
[-2.17217217],
[-2.15215215],
[-2.13213213],
[-2.11211211],

[-2.09209209],
[-2.07207207],
[-2.05205205],
[-2.03203203],
[-2.01201201],
[-1.99199199],
[-1.97197197],
[-1.95195195],
[-1.93193193],
[-1.91191191],
[-1.89189189],
[-1.87187187],
[-1.85185185],
[-1.83183183],
[-1.81181181],
[-1.79179179],
[-1.77177177],
[-1.75175175],
[-1.73173173],
[-1.71171171],
[-1.69169169],
[-1.67167167],
[-1.65165165],
[-1.63163163],
[-1.61161161],
[-1.59159159],
[-1.57157157],
[-1.55155155],
[-1.53153153],
[-1.51151151],
[-1.49149149],
[-1.47147147],
[-1.45145145],
[-1.43143143],
[-1.41141141],
[-1.39139139],
[-1.37137137],
[-1.35135135],
[-1.33133133],
[-1.31131131],
[-1.29129129],
[-1.27127127],
[-1.25125125],
[-1.23123123],
[-1.21121121],
[-1.19119119],
[-1.17117117],
[-1.15115115],
[-1.13113113],
[-1.11111111],
[-1.09109109],
[-1.07107107],
[-1.05105105],
[-1.03103103],
[-1.01101101],
[-0.99099099],

[-0.97097097],
[-0.95095095],
[-0.93093093],
[-0.91091091],
[-0.89089089],
[-0.87087087],
[-0.85085085],
[-0.83083083],
[-0.81081081],
[-0.79079079],
[-0.77077077],
[-0.75075075],
[-0.73073073],
[-0.71071071],
[-0.69069069],
[-0.67067067],
[-0.65065065],
[-0.63063063],
[-0.61061061],
[-0.59059059],
[-0.57057057],
[-0.55055055],
[-0.53053053],
[-0.51051051],
[-0.49049049],
[-0.47047047],
[-0.45045045],
[-0.43043043],
[-0.41041041],
[-0.39039039],
[-0.37037037],
[-0.35035035],
[-0.33033033],
[-0.31031031],
[-0.29029029],
[-0.27027027],
[-0.25025025],
[-0.23023023],
[-0.21021021],
[-0.19019019],
[-0.17017017],
[-0.15015015],
[-0.13013013],
[-0.11011011],
[-0.09009009],
[-0.07007007],
[-0.05005005],
[-0.03003003],
[-0.01001001],
[0.01001001],
[0.03003003],
[0.05005005],
[0.07007007],
[0.09009009],
[0.11011011],
[0.13013013],

[0.15015015],
[0.17017017],
[0.19019019],
[0.21021021],
[0.23023023],
[0.25025025],
[0.27027027],
[0.29029029],
[0.31031031],
[0.33033033],
[0.35035035],
[0.37037037],
[0.39039039],
[0.41041041],
[0.43043043],
[0.45045045],
[0.47047047],
[0.49049049],
[0.51051051],
[0.53053053],
[0.55055055],
[0.57057057],
[0.59059059],
[0.61061061],
[0.63063063],
[0.65065065],
[0.67067067],
[0.69069069],
[0.71071071],
[0.73073073],
[0.75075075],
[0.77077077],
[0.79079079],
[0.81081081],
[0.83083083],
[0.85085085],
[0.87087087],
[0.89089089],
[0.91091091],
[0.93093093],
[0.95095095],
[0.97097097],
[0.99099099],
[1.01101101],
[1.03103103],
[1.05105105],
[1.07107107],
[1.09109109],
[1.11111111],
[1.13113113],
[1.15115115],
[1.17117117],
[1.19119119],
[1.21121121],
[1.23123123],
[1.25125125],

[1.27127127],
[1.29129129],
[1.31131131],
[1.33133133],
[1.35135135],
[1.37137137],
[1.39139139],
[1.41141141],
[1.43143143],
[1.45145145],
[1.47147147],
[1.49149149],
[1.51151151],
[1.53153153],
[1.55155155],
[1.57157157],
[1.59159159],
[1.61161161],
[1.63163163],
[1.65165165],
[1.67167167],
[1.69169169],
[1.71171171],
[1.73173173],
[1.75175175],
[1.77177177],
[1.79179179],
[1.81181181],
[1.83183183],
[1.85185185],
[1.87187187],
[1.89189189],
[1.91191191],
[1.93193193],
[1.95195195],
[1.97197197],
[1.99199199],
[2.01201201],
[2.03203203],
[2.05205205],
[2.07207207],
[2.09209209],
[2.11211211],
[2.13213213],
[2.15215215],
[2.17217217],
[2.19219219],
[2.21221221],
[2.23223223],
[2.25225225],
[2.27227227],
[2.29229229],
[2.31231231],
[2.33233233],
[2.35235235],
[2.37237237],

[2.39239239],
[2.41241241],
[2.43243243],
[2.45245245],
[2.47247247],
[2.49249249],
[2.51251251],
[2.53253253],
[2.55255255],
[2.57257257],
[2.59259259],
[2.61261261],
[2.63263263],
[2.65265265],
[2.67267267],
[2.69269269],
[2.71271271],
[2.73273273],
[2.75275275],
[2.77277277],
[2.79279279],
[2.81281281],
[2.83283283],
[2.85285285],
[2.87287287],
[2.89289289],
[2.91291291],
[2.93293293],
[2.95295295],
[2.97297297],
[2.99299299],
[3.01301301],
[3.03303303],
[3.05305305],
[3.07307307],
[3.09309309],
[3.11311311],
[3.13313313],
[3.15315315],
[3.17317317],
[3.19319319],
[3.21321321],
[3.23323323],
[3.25325325],
[3.27327327],
[3.29329329],
[3.31331331],
[3.33333333],
[3.35335335],
[3.37337337],
[3.39339339],
[3.41341341],
[3.43343343],
[3.45345345],
[3.47347347],
[3.49349349],

[3.51351351],
[3.53353353],
[3.55355355],
[3.57357357],
[3.59359359],
[3.61361361],
[3.63363363],
[3.65365365],
[3.67367367],
[3.69369369],
[3.71371371],
[3.73373373],
[3.75375375],
[3.77377377],
[3.79379379],
[3.81381381],
[3.83383383],
[3.85385385],
[3.87387387],
[3.89389389],
[3.91391391],
[3.93393393],
[3.95395395],
[3.97397397],
[3.99399399],
[4.01401401],
[4.03403403],
[4.05405405],
[4.07407407],
[4.09409409],
[4.11411411],
[4.13413413],
[4.15415415],
[4.17417417],
[4.19419419],
[4.21421421],
[4.23423423],
[4.25425425],
[4.27427427],
[4.29429429],
[4.31431431],
[4.33433433],
[4.35435435],
[4.37437437],
[4.39439439],
[4.41441441],
[4.43443443],
[4.45445445],
[4.47447447],
[4.49449449],
[4.51451451],
[4.53453453],
[4.55455455],
[4.57457457],
[4.59459459],
[4.61461461],

[4.63463463],
[4.65465465],
[4.67467467],
[4.69469469],
[4.71471471],
[4.73473473],
[4.75475475],
[4.77477477],
[4.79479479],
[4.81481481],
[4.83483483],
[4.85485485],
[4.87487487],
[4.89489489],
[4.91491491],
[4.93493493],
[4.95495495],
[4.97497497],
[4.99499499],
[5.01501502],
[5.03503504],
[5.05505506],
[5.07507508],
[5.0950951],
[5.11511512],
[5.13513514],
[5.15515516],
[5.17517518],
[5.1951952],
[5.21521522],
[5.23523524],
[5.25525526],
[5.27527528],
[5.2952953],
[5.31531532],
[5.33533534],
[5.35535536],
[5.37537538],
[5.3953954],
[5.41541542],
[5.43543544],
[5.45545546],
[5.47547548],
[5.4954955],
[5.51551552],
[5.53553554],
[5.55555556],
[5.57557558],
[5.5955956],
[5.61561562],
[5.63563564],
[5.65565566],
[5.67567568],
[5.6956957],
[5.71571572],
[5.73573574],

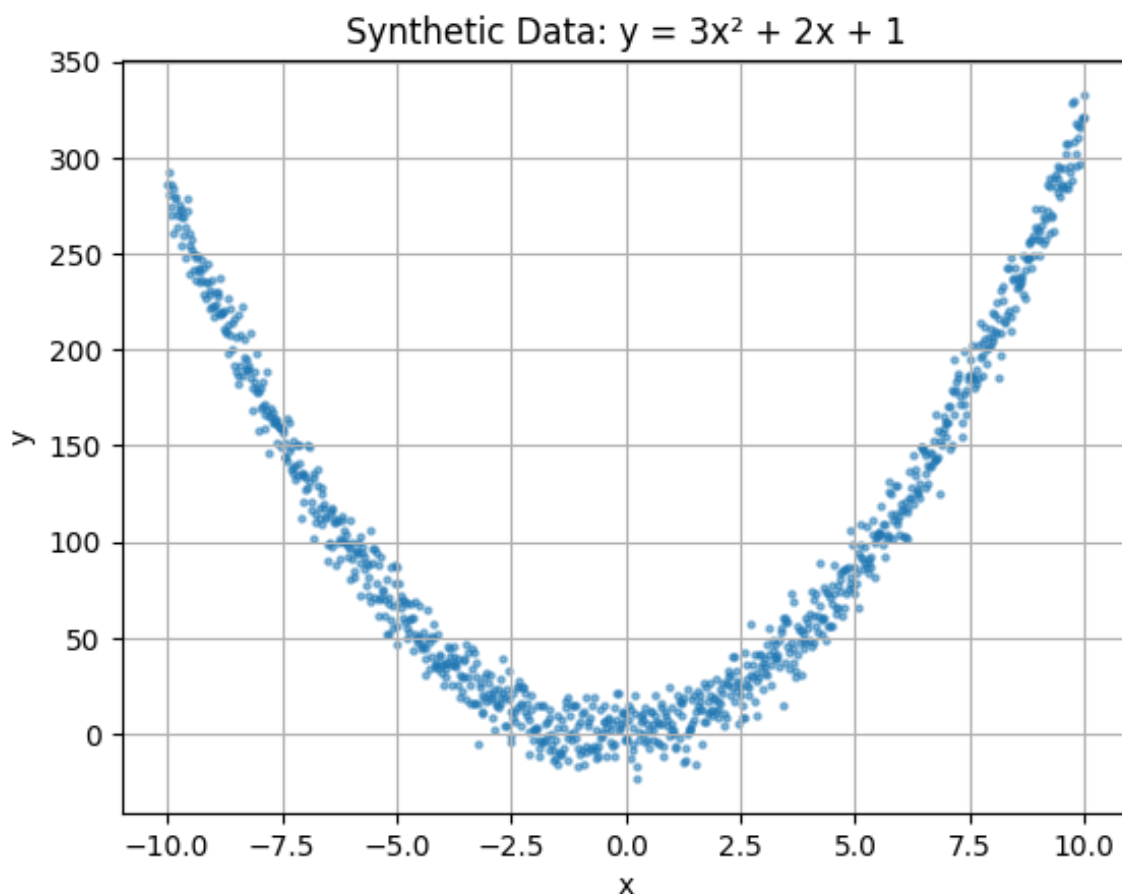
[5.75575576],
[5.77577578],
[5.7957958],
[5.81581582],
[5.83583584],
[5.85585586],
[5.87587588],
[5.8958959],
[5.91591592],
[5.93593594],
[5.95595596],
[5.97597598],
[5.995996],
[6.01601602],
[6.03603604],
[6.05605606],
[6.07607608],
[6.0960961],
[6.11611612],
[6.13613614],
[6.15615616],
[6.17617618],
[6.1961962],
[6.21621622],
[6.23623624],
[6.25625626],
[6.27627628],
[6.2962963],
[6.31631632],
[6.33633634],
[6.35635636],
[6.37637638],
[6.3963964],
[6.41641642],
[6.43643644],
[6.45645646],
[6.47647648],
[6.4964965],
[6.51651652],
[6.53653654],
[6.55655656],
[6.57657658],
[6.5965966],
[6.61661662],
[6.63663664],
[6.65665666],
[6.67667668],
[6.6966967],
[6.71671672],
[6.73673674],
[6.75675676],
[6.77677678],
[6.7967968],
[6.81681682],
[6.83683684],
[6.85685686],

[6.87687688],
[6.8968969],
[6.91691692],
[6.93693694],
[6.95695696],
[6.97697698],
[6.996997],
[7.01701702],
[7.03703704],
[7.05705706],
[7.07707708],
[7.0970971],
[7.11711712],
[7.13713714],
[7.15715716],
[7.17717718],
[7.1971972],
[7.21721722],
[7.23723724],
[7.25725726],
[7.27727728],
[7.2972973],
[7.31731732],
[7.33733734],
[7.35735736],
[7.37737738],
[7.3973974],
[7.41741742],
[7.43743744],
[7.45745746],
[7.47747748],
[7.4974975],
[7.51751752],
[7.53753754],
[7.55755756],
[7.57757758],
[7.5975976],
[7.61761762],
[7.63763764],
[7.65765766],
[7.67767768],
[7.6976977],
[7.71771772],
[7.73773774],
[7.75775776],
[7.77777778],
[7.7977978],
[7.81781782],
[7.83783784],
[7.85785786],
[7.87787788],
[7.8978979],
[7.91791792],
[7.93793794],
[7.95795796],
[7.97797798],

[7.997998],
[8.01801802],
[8.03803804],
[8.05805806],
[8.07807808],
[8.0980981],
[8.11811812],
[8.13813814],
[8.15815816],
[8.17817818],
[8.1981982],
[8.21821822],
[8.23823824],
[8.25825826],
[8.27827828],
[8.2982983],
[8.31831832],
[8.33833834],
[8.35835836],
[8.37837838],
[8.3983984],
[8.41841842],
[8.43843844],
[8.45845846],
[8.47847848],
[8.4984985],
[8.51851852],
[8.53853854],
[8.55855856],
[8.57857858],
[8.5985986],
[8.61861862],
[8.63863864],
[8.65865866],
[8.67867868],
[8.6986987],
[8.71871872],
[8.73873874],
[8.75875876],
[8.77877878],
[8.7987988],
[8.81881882],
[8.83883884],
[8.85885886],
[8.87887888],
[8.8988989],
[8.91891892],
[8.93893894],
[8.95895896],
[8.97897898],
[8.998999],
[9.01901902],
[9.03903904],
[9.05905906],
[9.07907908],
[9.0990991],

```
[ 9.11911912],  
[ 9.13913914],  
[ 9.15915916],  
[ 9.17917918],  
[ 9.1991992 ],  
[ 9.21921922],  
[ 9.23923924],  
[ 9.25925926],  
[ 9.27927928],  
[ 9.2992993 ],  
[ 9.31931932],  
[ 9.33933934],  
[ 9.35935936],  
[ 9.37937938],  
[ 9.3993994 ],  
[ 9.41941942],  
[ 9.43943944],  
[ 9.45945946],  
[ 9.47947948],  
[ 9.4994995 ],  
[ 9.51951952],  
[ 9.53953954],  
[ 9.55955956],  
[ 9.57957958],  
[ 9.5995996 ],  
[ 9.61961962],  
[ 9.63963964],  
[ 9.65965966],  
[ 9.67967968],  
[ 9.6996997 ],  
[ 9.71971972],  
[ 9.73973974],  
[ 9.75975976],  
[ 9.77977978],  
[ 9.7997998 ],  
[ 9.81981982],  
[ 9.83983984],  
[ 9.85985986],  
[ 9.87987988],  
[ 9.8998999 ],  
[ 9.91991992],  
[ 9.93993994],  
[ 9.95995996],  
[ 9.97997998],  
[ 10.          ]])
```

```
In [31]: plt.scatter(X, y, s=5, alpha=0.5)  
plt.title("Synthetic Data:  $y = 3x^2 + 2x + 1$ ")  
plt.xlabel("x")  
plt.ylabel("y")  
plt.grid(True)  
plt.show()
```



```
In [32]: X = MinMaxScaler(feature_range=(-1, 1)).fit_transform(X)
X
```

```
Out[32]: array([[ -1.          ],
                [-0.997998   ],
                [-0.995996   ],
                [-0.99399399],
                [-0.99199199],
                [-0.98998999],
                [-0.98798799],
                [-0.98598599],
                [-0.98398398],
                [-0.98198198],
                [-0.97997998],
                [-0.97797798],
                [-0.97597598],
                [-0.97397397],
                [-0.97197197],
                [-0.96996997],
                [-0.96796797],
                [-0.96596597],
                [-0.96396396],
                [-0.96196196],
                [-0.95995996],
                [-0.95795796],
                [-0.95595596],
                [-0.95395395],
                [-0.95195195],
                [-0.94994995],
```

[−0.94794795],
[−0.94594595],
[−0.94394394],
[−0.94194194],
[−0.93993994],
[−0.93793794],
[−0.93593594],
[−0.93393393],
[−0.93193193],
[−0.92992993],
[−0.92792793],
[−0.92592593],
[−0.92392392],
[−0.92192192],
[−0.91991992],
[−0.91791792],
[−0.91591592],
[−0.91391391],
[−0.91191191],
[−0.90990991],
[−0.90790791],
[−0.90590591],
[−0.9039039],
[−0.9019019],
[−0.8998999],
[−0.8978979],
[−0.8958959],
[−0.89389389],
[−0.89189189],
[−0.88988989],
[−0.88788789],
[−0.88588589],
[−0.88388388],
[−0.88188188],
[−0.87987988],
[−0.87787788],
[−0.87587588],
[−0.87387387],
[−0.87187187],
[−0.86986987],
[−0.86786787],
[−0.86586587],
[−0.86386386],
[−0.86186186],
[−0.85985986],
[−0.85785786],
[−0.85585586],
[−0.85385385],
[−0.85185185],
[−0.84984985],
[−0.84784785],
[−0.84584585],
[−0.84384384],
[−0.84184184],
[−0.83983984],
[−0.83783784],

[−0.83583584],
[−0.83383383],
[−0.83183183],
[−0.82982983],
[−0.82782783],
[−0.82582583],
[−0.82382382],
[−0.82182182],
[−0.81981982],
[−0.81781782],
[−0.81581582],
[−0.81381381],
[−0.81181181],
[−0.80980981],
[−0.80780781],
[−0.80580581],
[−0.8038038],
[−0.8018018],
[−0.7997998],
[−0.7977978],
[−0.7957958],
[−0.79379379],
[−0.79179179],
[−0.78978979],
[−0.78778779],
[−0.78578579],
[−0.78378378],
[−0.78178178],
[−0.77977978],
[−0.77777778],
[−0.77577578],
[−0.77377377],
[−0.77177177],
[−0.76976977],
[−0.76776777],
[−0.76576577],
[−0.76376376],
[−0.76176176],
[−0.75975976],
[−0.75775776],
[−0.75575576],
[−0.75375375],
[−0.75175175],
[−0.74974975],
[−0.74774775],
[−0.74574575],
[−0.74374374],
[−0.74174174],
[−0.73973974],
[−0.73773774],
[−0.73573574],
[−0.73373373],
[−0.73173173],
[−0.72972973],
[−0.72772773],
[−0.72572573],

[−0.72372372],
[−0.72172172],
[−0.71971972],
[−0.71771772],
[−0.71571572],
[−0.71371371],
[−0.71171171],
[−0.70970971],
[−0.70770771],
[−0.70570571],
[−0.7037037],
[−0.7017017],
[−0.6996997],
[−0.6976977],
[−0.6956957],
[−0.69369369],
[−0.69169169],
[−0.68968969],
[−0.68768769],
[−0.68568569],
[−0.68368368],
[−0.68168168],
[−0.67967968],
[−0.67767768],
[−0.67567568],
[−0.67367367],
[−0.67167167],
[−0.66966967],
[−0.66766767],
[−0.66566567],
[−0.66366366],
[−0.66166166],
[−0.65965966],
[−0.65765766],
[−0.65565566],
[−0.65365365],
[−0.65165165],
[−0.64964965],
[−0.64764765],
[−0.64564565],
[−0.64364364],
[−0.64164164],
[−0.63963964],
[−0.63763764],
[−0.63563564],
[−0.63363363],
[−0.63163163],
[−0.62962963],
[−0.62762763],
[−0.62562563],
[−0.62362362],
[−0.62162162],
[−0.61961962],
[−0.61761762],
[−0.61561562],
[−0.61361361],

[−0.61161161],
[−0.60960961],
[−0.60760761],
[−0.60560561],
[−0.6036036],
[−0.6016016],
[−0.5995996],
[−0.5975976],
[−0.5955956],
[−0.59359359],
[−0.59159159],
[−0.58958959],
[−0.58758759],
[−0.58558559],
[−0.58358358],
[−0.58158158],
[−0.57957958],
[−0.57757758],
[−0.57557558],
[−0.57357357],
[−0.57157157],
[−0.56956957],
[−0.56756757],
[−0.56556557],
[−0.56356356],
[−0.56156156],
[−0.55955956],
[−0.55755756],
[−0.55555556],
[−0.55355355],
[−0.55155155],
[−0.54954955],
[−0.54754755],
[−0.54554555],
[−0.54354354],
[−0.54154154],
[−0.53953954],
[−0.53753754],
[−0.53553554],
[−0.53353353],
[−0.53153153],
[−0.52952953],
[−0.52752753],
[−0.52552553],
[−0.52352352],
[−0.52152152],
[−0.51951952],
[−0.51751752],
[−0.51551552],
[−0.51351351],
[−0.51151151],
[−0.50950951],
[−0.50750751],
[−0.50550551],
[−0.5035035],
[−0.5015015],

[-0.4994995],
[-0.4974975],
[-0.4954955],
[-0.49349349],
[-0.49149149],
[-0.48948949],
[-0.48748749],
[-0.48548549],
[-0.48348348],
[-0.48148148],
[-0.47947948],
[-0.47747748],
[-0.47547548],
[-0.47347347],
[-0.47147147],
[-0.46946947],
[-0.46746747],
[-0.46546547],
[-0.46346346],
[-0.46146146],
[-0.45945946],
[-0.45745746],
[-0.45545546],
[-0.45345345],
[-0.45145145],
[-0.44944945],
[-0.44744745],
[-0.44544545],
[-0.44344344],
[-0.44144144],
[-0.43943944],
[-0.43743744],
[-0.43543544],
[-0.43343343],
[-0.43143143],
[-0.42942943],
[-0.42742743],
[-0.42542543],
[-0.42342342],
[-0.42142142],
[-0.41941942],
[-0.41741742],
[-0.41541542],
[-0.41341341],
[-0.41141141],
[-0.40940941],
[-0.40740741],
[-0.40540541],
[-0.4034034],
[-0.4014014],
[-0.3993994],
[-0.3973974],
[-0.3953954],
[-0.39339339],
[-0.39139139],
[-0.38938939],

[−0.38738739],
[−0.38538539],
[−0.38338338],
[−0.38138138],
[−0.37937938],
[−0.37737738],
[−0.37537538],
[−0.37337337],
[−0.37137137],
[−0.36936937],
[−0.36736737],
[−0.36536537],
[−0.36336336],
[−0.36136136],
[−0.35935936],
[−0.35735736],
[−0.35535536],
[−0.35335335],
[−0.35135135],
[−0.34934935],
[−0.34734735],
[−0.34534535],
[−0.34334334],
[−0.34134134],
[−0.33933934],
[−0.33733734],
[−0.33533534],
[−0.33333333],
[−0.33133133],
[−0.32932933],
[−0.32732733],
[−0.32532533],
[−0.32332332],
[−0.32132132],
[−0.31931932],
[−0.31731732],
[−0.31531532],
[−0.31331331],
[−0.31131131],
[−0.30930931],
[−0.30730731],
[−0.30530531],
[−0.3033033],
[−0.3013013],
[−0.2992993],
[−0.2972973],
[−0.2952953],
[−0.29329329],
[−0.29129129],
[−0.28928929],
[−0.28728729],
[−0.28528529],
[−0.28328328],
[−0.28128128],
[−0.27927928],
[−0.27727728],

[−0.27527528],
[−0.27327327],
[−0.27127127],
[−0.26926927],
[−0.26726727],
[−0.26526527],
[−0.26326326],
[−0.26126126],
[−0.25925926],
[−0.25725726],
[−0.25525526],
[−0.25325325],
[−0.25125125],
[−0.24924925],
[−0.24724725],
[−0.24524525],
[−0.24324324],
[−0.24124124],
[−0.23923924],
[−0.23723724],
[−0.23523524],
[−0.23323323],
[−0.23123123],
[−0.22922923],
[−0.22722723],
[−0.22522523],
[−0.22322322],
[−0.22122122],
[−0.21921922],
[−0.21721722],
[−0.21521522],
[−0.21321321],
[−0.21121121],
[−0.20920921],
[−0.20720721],
[−0.20520521],
[−0.2032032],
[−0.2012012],
[−0.1991992],
[−0.1971972],
[−0.1951952],
[−0.19319319],
[−0.19119119],
[−0.18918919],
[−0.18718719],
[−0.18518519],
[−0.18318318],
[−0.18118118],
[−0.17917918],
[−0.17717718],
[−0.17517518],
[−0.17317317],
[−0.17117117],
[−0.16916917],
[−0.16716717],
[−0.16516517],

[-0.16316316],
[-0.16116116],
[-0.15915916],
[-0.15715716],
[-0.15515516],
[-0.15315315],
[-0.15115115],
[-0.14914915],
[-0.14714715],
[-0.14514515],
[-0.14314314],
[-0.14114114],
[-0.13913914],
[-0.13713714],
[-0.13513514],
[-0.13313313],
[-0.13113113],
[-0.12912913],
[-0.12712713],
[-0.12512513],
[-0.12312312],
[-0.12112112],
[-0.11911912],
[-0.11711712],
[-0.11511512],
[-0.11311311],
[-0.11111111],
[-0.10910911],
[-0.10710711],
[-0.10510511],
[-0.1031031],
[-0.1011011],
[-0.0990991],
[-0.0970971],
[-0.0950951],
[-0.09309309],
[-0.09109109],
[-0.08908909],
[-0.08708709],
[-0.08508509],
[-0.08308308],
[-0.08108108],
[-0.07907908],
[-0.07707708],
[-0.07507508],
[-0.07307307],
[-0.07107107],
[-0.06906907],
[-0.06706707],
[-0.06506507],
[-0.06306306],
[-0.06106106],
[-0.05905906],
[-0.05705706],
[-0.05505506],
[-0.05305305],

[-0.05105105],
[-0.04904905],
[-0.04704705],
[-0.04504505],
[-0.04304304],
[-0.04104104],
[-0.03903904],
[-0.03703704],
[-0.03503504],
[-0.03303303],
[-0.03103103],
[-0.02902903],
[-0.02702703],
[-0.02502503],
[-0.02302302],
[-0.02102102],
[-0.01901902],
[-0.01701702],
[-0.01501502],
[-0.01301301],
[-0.01101101],
[-0.00900901],
[-0.00700701],
[-0.00500501],
[-0.003003],
[-0.001001],
[0.001001],
[0.003003],
[0.00500501],
[0.00700701],
[0.00900901],
[0.01101101],
[0.01301301],
[0.01501502],
[0.01701702],
[0.01901902],
[0.02102102],
[0.02302302],
[0.02502503],
[0.02702703],
[0.02902903],
[0.03103103],
[0.03303303],
[0.03503504],
[0.03703704],
[0.03903904],
[0.04104104],
[0.04304304],
[0.04504505],
[0.04704705],
[0.04904905],
[0.05105105],
[0.05305305],
[0.05505506],
[0.05705706],
[0.05905906],

[0.06106106],
[0.06306306],
[0.06506507],
[0.06706707],
[0.06906907],
[0.07107107],
[0.07307307],
[0.07507508],
[0.07707708],
[0.07907908],
[0.08108108],
[0.08308308],
[0.08508509],
[0.08708709],
[0.08908909],
[0.09109109],
[0.09309309],
[0.0950951],
[0.0970971],
[0.0990991],
[0.1011011],
[0.1031031],
[0.10510511],
[0.10710711],
[0.10910911],
[0.11111111],
[0.11311311],
[0.11511512],
[0.11711712],
[0.11911912],
[0.12112112],
[0.12312312],
[0.12512513],
[0.12712713],
[0.12912913],
[0.13113113],
[0.13313313],
[0.13513514],
[0.13713714],
[0.13913914],
[0.14114114],
[0.14314314],
[0.14514515],
[0.14714715],
[0.14914915],
[0.15115115],
[0.15315315],
[0.15515516],
[0.15715716],
[0.15915916],
[0.16116116],
[0.16316316],
[0.16516517],
[0.16716717],
[0.16916917],
[0.17117117],

[0.17317317],
[0.17517518],
[0.17717718],
[0.17917918],
[0.18118118],
[0.18318318],
[0.18518519],
[0.18718719],
[0.18918919],
[0.19119119],
[0.19319319],
[0.1951952],
[0.1971972],
[0.1991992],
[0.2012012],
[0.2032032],
[0.20520521],
[0.20720721],
[0.20920921],
[0.21121121],
[0.21321321],
[0.21521522],
[0.21721722],
[0.21921922],
[0.22122122],
[0.22322322],
[0.22522523],
[0.22722723],
[0.22922923],
[0.23123123],
[0.23323323],
[0.23523524],
[0.23723724],
[0.23923924],
[0.24124124],
[0.24324324],
[0.24524525],
[0.24724725],
[0.24924925],
[0.25125125],
[0.25325325],
[0.25525526],
[0.25725726],
[0.25925926],
[0.26126126],
[0.26326326],
[0.26526527],
[0.26726727],
[0.26926927],
[0.27127127],
[0.27327327],
[0.27527528],
[0.27727728],
[0.27927928],
[0.28128128],
[0.28328328],

[0.28528529],
[0.28728729],
[0.28928929],
[0.29129129],
[0.29329329],
[0.2952953],
[0.2972973],
[0.2992993],
[0.3013013],
[0.3033033],
[0.30530531],
[0.30730731],
[0.30930931],
[0.31131131],
[0.31331331],
[0.31531532],
[0.31731732],
[0.31931932],
[0.32132132],
[0.32332332],
[0.32532533],
[0.32732733],
[0.32932933],
[0.33133133],
[0.33333333],
[0.33533534],
[0.33733734],
[0.33933934],
[0.34134134],
[0.34334334],
[0.34534535],
[0.34734735],
[0.34934935],
[0.35135135],
[0.35335335],
[0.35535536],
[0.35735736],
[0.35935936],
[0.36136136],
[0.36336336],
[0.36536537],
[0.36736737],
[0.36936937],
[0.37137137],
[0.37337337],
[0.37537538],
[0.37737738],
[0.37937938],
[0.38138138],
[0.38338338],
[0.38538539],
[0.38738739],
[0.38938939],
[0.39139139],
[0.39339339],
[0.3953954],

[0.3973974],
[0.3993994],
[0.4014014],
[0.4034034],
[0.4054054],
[0.4074074],
[0.4094094],
[0.4114114],
[0.4134134],
[0.4154154],
[0.4174174],
[0.4194194],
[0.4214214],
[0.4234234],
[0.4254254],
[0.4274274],
[0.4294294],
[0.4314314],
[0.4334334],
[0.4354354],
[0.4374374],
[0.4394394],
[0.4414414],
[0.4434434],
[0.4454454],
[0.4474474],
[0.4494494],
[0.4514514],
[0.4534534],
[0.4554554],
[0.4574574],
[0.4594594],
[0.4614614],
[0.4634634],
[0.4654654],
[0.4674674],
[0.4694694],
[0.4714714],
[0.4734734],
[0.4754754],
[0.4774774],
[0.4794794],
[0.4814814],
[0.4834834],
[0.4854854],
[0.4874874],
[0.4894894],
[0.4914914],
[0.4934934],
[0.4954955],
[0.4974975],
[0.4994995],
[0.5015015],
[0.5035035],
[0.5055055],
[0.5075075],

[0.50950951],
[0.51151151],
[0.51351351],
[0.51551552],
[0.51751752],
[0.51951952],
[0.52152152],
[0.52352352],
[0.52552553],
[0.52752753],
[0.52952953],
[0.53153153],
[0.53353353],
[0.53553554],
[0.53753754],
[0.53953954],
[0.54154154],
[0.54354354],
[0.54554555],
[0.54754755],
[0.54954955],
[0.55155155],
[0.55355355],
[0.55555556],
[0.55755756],
[0.55955956],
[0.56156156],
[0.56356356],
[0.56556557],
[0.56756757],
[0.56956957],
[0.57157157],
[0.57357357],
[0.57557558],
[0.57757758],
[0.57957958],
[0.58158158],
[0.58358358],
[0.58558559],
[0.58758759],
[0.58958959],
[0.59159159],
[0.59359359],
[0.59559556],
[0.5975976],
[0.5995996],
[0.6016016],
[0.6036036],
[0.60560561],
[0.60760761],
[0.60960961],
[0.61161161],
[0.61361361],
[0.61561562],
[0.61761762],
[0.61961962],

[0.62162162],
[0.62362362],
[0.62562563],
[0.62762763],
[0.62962963],
[0.63163163],
[0.63363363],
[0.63563564],
[0.63763764],
[0.63963964],
[0.64164164],
[0.64364364],
[0.64564565],
[0.64764765],
[0.64964965],
[0.65165165],
[0.65365365],
[0.65565566],
[0.65765766],
[0.65965966],
[0.66166166],
[0.66366366],
[0.66566567],
[0.66766767],
[0.66966967],
[0.67167167],
[0.67367367],
[0.67567568],
[0.67767768],
[0.67967968],
[0.68168168],
[0.68368368],
[0.68568569],
[0.68768769],
[0.68968969],
[0.69169169],
[0.69369369],
[0.6956957],
[0.6976977],
[0.6996997],
[0.7017017],
[0.7037037],
[0.70570571],
[0.70770771],
[0.70970971],
[0.71171171],
[0.71371371],
[0.71571572],
[0.71771772],
[0.71971972],
[0.72172172],
[0.72372372],
[0.72572573],
[0.72772773],
[0.72972973],
[0.73173173],

[0.73373373],
[0.73573574],
[0.73773774],
[0.73973974],
[0.74174174],
[0.74374374],
[0.74574575],
[0.74774775],
[0.74974975],
[0.75175175],
[0.75375375],
[0.75575576],
[0.75775776],
[0.75975976],
[0.76176176],
[0.76376376],
[0.76576577],
[0.76776777],
[0.76976977],
[0.77177177],
[0.77377377],
[0.77577578],
[0.77777778],
[0.77977978],
[0.78178178],
[0.78378378],
[0.78578579],
[0.78778779],
[0.78978979],
[0.79179179],
[0.79379379],
[0.7957958],
[0.7977978],
[0.7997998],
[0.8018018],
[0.8038038],
[0.80580581],
[0.80780781],
[0.80980981],
[0.81181181],
[0.81381381],
[0.81581582],
[0.81781782],
[0.81981982],
[0.82182182],
[0.82382382],
[0.82582583],
[0.82782783],
[0.82982983],
[0.83183183],
[0.83383383],
[0.83583584],
[0.83783784],
[0.83983984],
[0.84184184],
[0.84384384],

[0.84584585],
[0.84784785],
[0.84984985],
[0.85185185],
[0.85385385],
[0.85585586],
[0.85785786],
[0.85985986],
[0.86186186],
[0.86386386],
[0.86586587],
[0.86786787],
[0.86986987],
[0.87187187],
[0.87387387],
[0.87587588],
[0.87787788],
[0.87987988],
[0.88188188],
[0.88388388],
[0.88588589],
[0.88788789],
[0.88988989],
[0.89189189],
[0.89389389],
[0.8958959],
[0.8978979],
[0.8998999],
[0.9019019],
[0.9039039],
[0.90590591],
[0.90790791],
[0.90990991],
[0.91191191],
[0.91391391],
[0.91591592],
[0.91791792],
[0.91991992],
[0.92192192],
[0.92392392],
[0.92592593],
[0.92792793],
[0.92992993],
[0.93193193],
[0.93393393],
[0.93593594],
[0.93793794],
[0.93993994],
[0.94194194],
[0.94394394],
[0.94594595],
[0.94794795],
[0.94994995],
[0.95195195],
[0.95395395],
[0.95595596],

```
[ 0.95795796],
[ 0.95995996],
[ 0.96196196],
[ 0.96396396],
[ 0.96596597],
[ 0.96796797],
[ 0.96996997],
[ 0.97197197],
[ 0.97397397],
[ 0.97597598],
[ 0.97797798],
[ 0.97997998],
[ 0.98198198],
[ 0.98398398],
[ 0.98598599],
[ 0.98798799],
[ 0.98998999],
[ 0.99199199],
[ 0.99399399],
[ 0.995996  ],
[ 0.997998  ],
[ 1.         ]])
```

```
In [33]: import time
```

```
In [34]: optimizers = {
          'SGD': tf.keras.optimizers.SGD(learning_rate=0.1),
          'SGD+Momentum': tf.keras.optimizers.SGD(learning_rate=0.03, mom
          'RMSprop': tf.keras.optimizers.RMSprop(learning_rate=0.003),
          'Adam': tf.keras.optimizers.Adam(learning_rate=0.001)
        }
```

```
In [35]: def train_with_optimizer(name, optimizer):
          model = models.Sequential([
              layers.Input(shape = (1,)),
              layers.Dense(16, activation = 'relu'),
              layers.Dense(8, activation = 'relu'),
              layers.Dense(1, activation = 'linear')
          ])

          model.compile(optimizer=optimizer, loss='mse')

          start = time.time()
          history = model.fit(X, y, epochs=100, batch_size=32, verbose=0)
          duration = time.time() - start

          final_loss = history.history['loss'][-1]

          print(f"{name} - Final Loss: {final_loss:.4f}, Time: {duration:}

          return history.history['loss'], final_loss, duration, model
```

```
In [36]: loss_histories = {}
          final_losses = {}
          train_times = {}
```

```

trained_models = {}

for name, opt in optimizers.items():
    print(f"Training with {name}...")
    loss, final_loss, duration, model = train_with_optimizer(name,

    loss_histories[name] = loss
    final_losses[name] = final_loss
    train_times[name] = duration
    trained_models[name] = model

```

Training with SGD...

SGD – Final Loss: 8347.5322, Time: 14.74s

Training with SGD+Momentum...

SGD+Momentum – Final Loss: 8332.3301, Time: 12.22s

Training with RMSprop...

RMSprop – Final Loss: 103.2163, Time: 12.78s

Training with Adam...

Adam – Final Loss: 6433.3408, Time: 13.39s

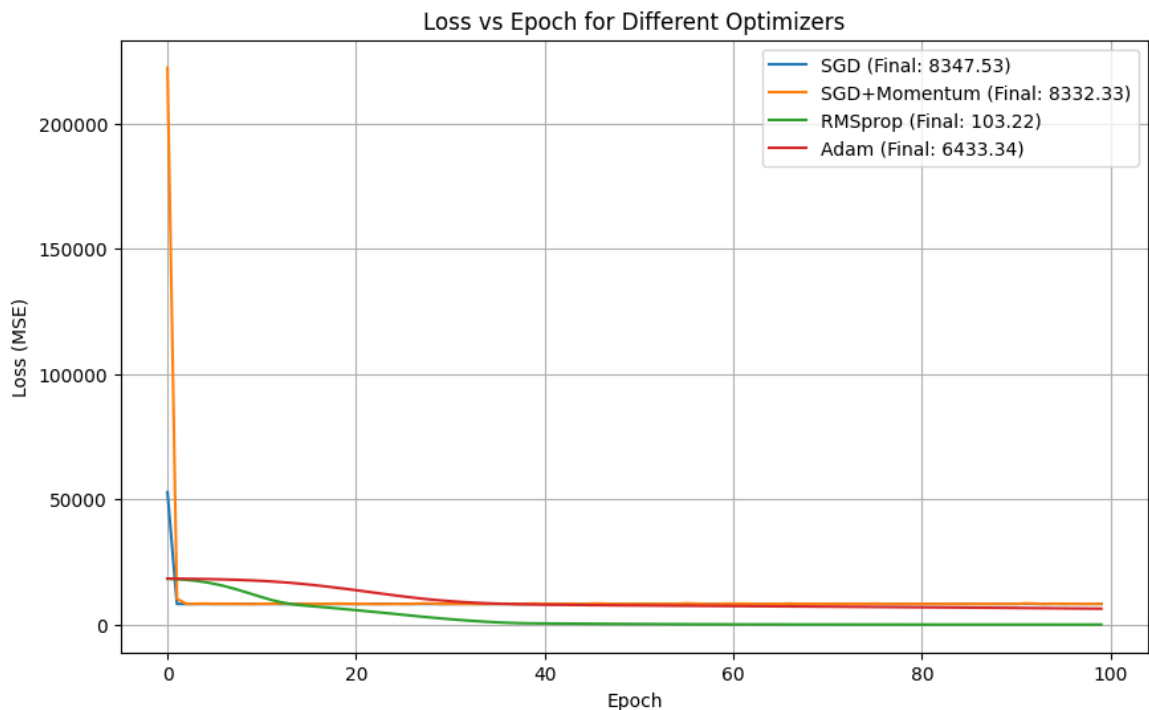
```

In [37]: plt.figure(figsize=(10, 6))

for name, loss in loss_histories.items():
    plt.plot(loss, label=f"{name} (Final: {final_losses[name]:.2f})")

plt.xlabel("Epoch")
plt.ylabel("Loss (MSE)")
plt.title("Loss vs Epoch for Different Optimizers")
plt.legend()
plt.grid(True)
plt.show()

```



```

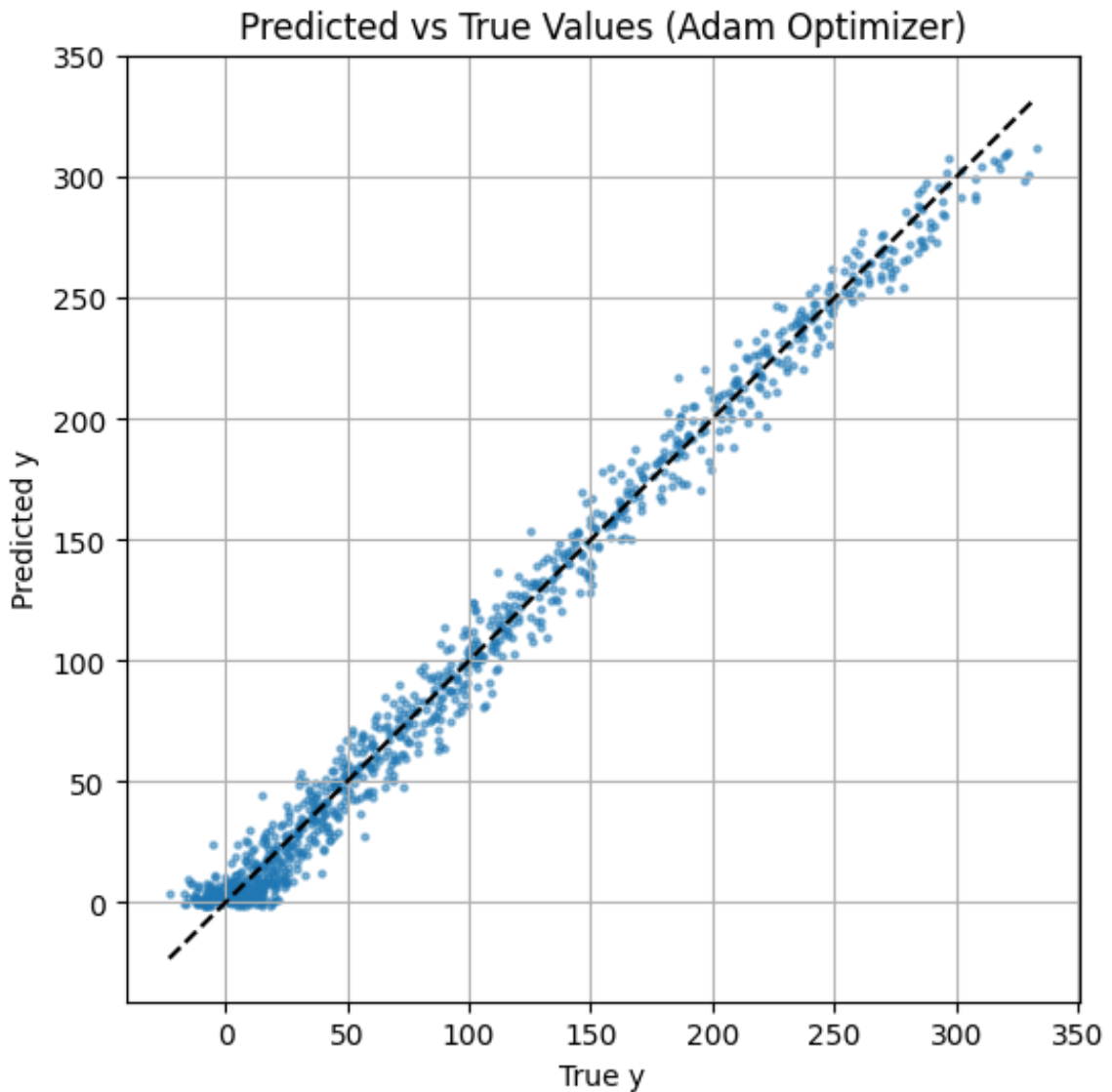
In [38]: y_pred = trained_models['RMSprop'].predict(X)

plt.figure(figsize=(6, 6))
plt.scatter(y, y_pred, alpha=0.5, s=5)

```

```
plt.xlabel("True y")
plt.ylabel("Predicted y")
plt.title("Predicted vs True Values (Adam Optimizer)")
plt.grid(True)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'k--') # y = x line
plt.show()
```

32/32 ————— 0s 5ms/step



Question 4

You're working with a financial tech company building an automated check-processing system. One key task is to read handwritten numeric account codes from scanned checks. The system must be both highly accurate and computationally efficient.

To validate different neural network strategies, you're required to implement a digit classification system using the MNIST dataset (digits 0–9, 10 classes) using two approaches:

- A deep learning framework (e.g., PyTorch, TensorFlow) and use

CrossEntropyLoss and Adam optimizer

- Manual implementation using NumPy without any deep learning framework

Train a simple feedforward ANN on the MNIST dataset (10 classes) using both approaches and compare their behavior.

```
In [39]: from tensorflow.keras.datasets import mnist
        from tensorflow.keras.utils import to_categorical
```

```
In [40]: (X_train, y_train), (X_test, y_test) = mnist.load_data()
```

```
In [41]: (X_train, y_train), (X_test, y_test) = mnist.load_data()
```

```
In [42]: X_train.shape
```

```
Out[42]: (60000, 28, 28)
```

```
In [43]: y_train.shape
```

```
Out[43]: (60000,)
```

```
In [44]: X_train = X_train.reshape(-1, 28*28).astype("float32") / 255.0
        X_test = X_test.reshape(-1, 28*28).astype("float32") / 255.0
```

```
In [45]: X_train.shape
```

```
Out[45]: (60000, 784)
```

```
In [46]: 28*28
```

```
Out[46]: 784
```

```
In [47]: model = models.Sequential([
        layers.Input(shape=(784,)),
        layers.Dense(128, activation='relu'),
        layers.Dense(64, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
```

```
In [48]: model.compile(
        optimizer = 'adam',
        loss = 'sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
```

```
In [49]: history = model.fit(X_train, y_train, epochs=10, batch_size=64, val
```

```

Epoch 1/10
938/938 ————— 7s 5ms/step – accuracy: 0.8508 – loss:
0.5039 – val_accuracy: 0.9593 – val_loss: 0.1390
Epoch 2/10
938/938 ————— 2s 3ms/step – accuracy: 0.9628 – loss:
0.1232 – val_accuracy: 0.9654 – val_loss: 0.1086
Epoch 3/10
938/938 ————— 2s 3ms/step – accuracy: 0.9750 – loss:
0.0796 – val_accuracy: 0.9750 – val_loss: 0.0854
Epoch 4/10
938/938 ————— 3s 3ms/step – accuracy: 0.9841 – loss:
0.0546 – val_accuracy: 0.9769 – val_loss: 0.0744
Epoch 5/10
938/938 ————— 3s 3ms/step – accuracy: 0.9861 – loss:
0.0443 – val_accuracy: 0.9754 – val_loss: 0.0813
Epoch 6/10
938/938 ————— 2s 3ms/step – accuracy: 0.9896 – loss:
0.0338 – val_accuracy: 0.9794 – val_loss: 0.0814
Epoch 7/10
938/938 ————— 3s 3ms/step – accuracy: 0.9911 – loss:
0.0271 – val_accuracy: 0.9765 – val_loss: 0.0839
Epoch 8/10
938/938 ————— 3s 3ms/step – accuracy: 0.9917 – loss:
0.0260 – val_accuracy: 0.9781 – val_loss: 0.0762
Epoch 9/10
938/938 ————— 3s 4ms/step – accuracy: 0.9940 – loss:
0.0206 – val_accuracy: 0.9795 – val_loss: 0.0746
Epoch 10/10
938/938 ————— 4s 3ms/step – accuracy: 0.9954 – loss:
0.0158 – val_accuracy: 0.9733 – val_loss: 0.1001

```

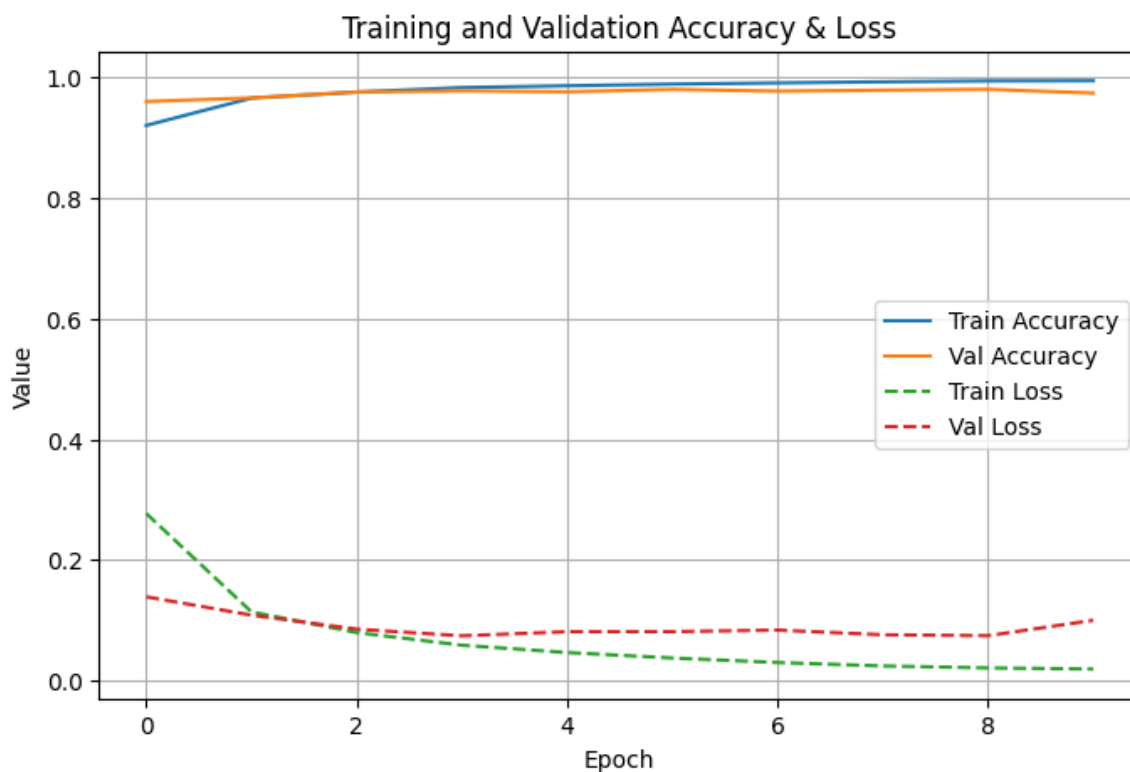
```
In [50]: test_acc = model.evaluate(X_test, y_test, verbose=0)[1]
```

```
In [51]: plt.figure(figsize=(8, 5))

# Accuracy
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')

# Loss
plt.plot(history.history['loss'], label='Train Loss', linestyle='--')
plt.plot(history.history['val_loss'], label='Val Loss', linestyle='--')

plt.title("Training and Validation Accuracy & Loss")
plt.xlabel("Epoch")
plt.ylabel("Value")
plt.legend()
plt.grid(True)
plt.show()
```



numpy only

```
In [52]: def one_hot(y, num_classes=10):
          return np.eye(num_classes)[y]
```

```
y_train_oh = one_hot(y_train)
y_test_oh = one_hot(y_test)
```

```
In [53]: y_train_oh
```

```
Out[53]: array([[0., 0., 0., ..., 0., 0., 0.],
                [1., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                ...,
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 0., 0.],
                [0., 0., 0., ..., 0., 1., 0.]])
```

```
In [54]: def init_weights():
          np.random.seed(42)

          W1 = np.random.randn(784, 128) * 0.01
          b1 = np.zeros((1, 128))

          W2 = np.random.randn(128, 64) * 0.01
          b2 = np.zeros((1, 64))

          W3 = np.random.randn(64, 10) * 0.01
          b3 = np.zeros((1, 10))

          return W1, b1, W2, b2, W3, b3
```

```
In [55]: def relu(z):
          return np.maximum(0, z)

          def relu_derivative(z):
              return (z > 0).astype(float)

          def softmax(z):
              e_z = np.exp(z - np.max(z, axis=1, keepdims=True))
              return e_z / np.sum(e_z, axis=1, keepdims=True)
```

```
In [56]: def forward(X, W1, b1, W2, b2, W3, b3):
          Z1 = X @ W1 + b1
          A1 = relu(Z1)

          Z2 = A1 @ W2 + b2
          A2 = relu(Z2)

          Z3 = A2 @ W3 + b3
          A3 = softmax(Z3)

          cache = (X, Z1, A1, Z2, A2, Z3, A3)
          return A3, cache
          # @ is used for matrix mul
```

```
In [57]: def compute_loss(y_true, y_pred):
          m = y_true.shape[0]
          log_probs = -np.log(y_pred + 1e-8)
          loss = np.sum(y_true * log_probs) / m
          return loss
```

```
In [58]: def backward(y_true, cache, W1, b1, W2, b2, W3, b3, learning_rate):
          X, Z1, A1, Z2, A2, Z3, A3 = cache
          m = y_true.shape[0]

          # Output layer
          dZ3 = A3 - y_true # shape (m, 10)
          dW3 = A2.T @ dZ3 / m
          db3 = np.sum(dZ3, axis=0, keepdims=True) / m

          # Layer 2
          dA2 = dZ3 @ W3.T
          dZ2 = dA2 * relu_derivative(Z2)
          dW2 = A1.T @ dZ2 / m
          db2 = np.sum(dZ2, axis=0, keepdims=True) / m

          # Layer 1
          dA1 = dZ2 @ W2.T
          dZ1 = dA1 * relu_derivative(Z1)
          dW1 = X.T @ dZ1 / m
          db1 = np.sum(dZ1, axis=0, keepdims=True) / m

          # Gradient descent update
          W3 -= learning_rate * dW3
          b3 -= learning_rate * db3
          W2 -= learning_rate * dW2
```

```

b2 -= learning_rate * db2
W1 -= learning_rate * dW1
b1 -= learning_rate * db1

return W1, b1, W2, b2, W3, b3

```

```

In [59]: W1, b1, W2, b2, W3, b3 = init_weights()

epochs = 10
batch_size = 64
learning_rate = 0.1

losses = []

for epoch in range(epochs):
    perm = np.random.permutation(X_train.shape[0])
    X_train_shuffled = X_train[perm]
    y_train_oh_shuffled = y_train_oh[perm]

    for i in range(0, X_train.shape[0], batch_size):
        X_batch = X_train_shuffled[i:i+batch_size]
        y_batch = y_train_oh_shuffled[i:i+batch_size]

        y_pred, cache = forward(X_batch, W1, b1, W2, b2, W3, b3)
        loss = compute_loss(y_batch, y_pred)
        W1, b1, W2, b2, W3, b3 = backward(y_batch, cache, W1, b1, W2, b2, W3, b3)

    # Loss on full training set
    y_pred_all, _ = forward(X_train, W1, b1, W2, b2, W3, b3)
    epoch_loss = compute_loss(y_train_oh, y_pred_all)
    losses.append(epoch_loss)

    print(f"Epoch {epoch+1}/{epochs} - Loss: {epoch_loss:.4f}")

```

```

Epoch 1/10 - Loss: 0.4401
Epoch 2/10 - Loss: 0.2513
Epoch 3/10 - Loss: 0.1498
Epoch 4/10 - Loss: 0.1441
Epoch 5/10 - Loss: 0.0991
Epoch 6/10 - Loss: 0.0719
Epoch 7/10 - Loss: 0.0582
Epoch 8/10 - Loss: 0.0493
Epoch 9/10 - Loss: 0.0536
Epoch 10/10 - Loss: 0.0454

```

```

In [60]: y_pred_test, _ = forward(X_test, W1, b1, W2, b2, W3, b3)

y_pred_labels = np.argmax(y_pred_test, axis=1)

accuracy = np.mean(y_pred_labels == y_test)
print(f"Test Accuracy (NumPy model): {accuracy:.4f}")

```

```
Test Accuracy (NumPy model): 0.9721
```

```

In [62]: print("Accuracy Comparison")
print(f"TensorFlow model accuracy: {test_acc:.4f}")

```

```
print(f"NumPy-only model accuracy:    {accuracy:.4f}")

print("\n Summary")
print("TensorFlow:")
print("- High-level API with automatic backpropagation")

print("- Uses Adam optimizer with adaptive learning rate")
print("- Fast convergence")

print("\nNumPy-only:")
print("- Manual forward pass, backpropagation, and updates")

print("- Simple SGD optimizer with fixed learning rate")

print("- Good accuracy but harder to tune")
```

Accuracy Comparison

TensorFlow model accuracy: 0.9733
NumPy-only model accuracy: 0.9721

Summary

TensorFlow:

- High-level API with automatic backpropagation
- Uses Adam optimizer with adaptive learning rate
- Fast convergence

NumPy-only:

- Manual forward pass, backpropagation, and updates
- Simple SGD optimizer with fixed learning rate
- Good accuracy but harder to tune

In [61]: